

dzenmcxai

October 24, 2024

```
[1]: pip install tensorflow keras
```

```
Requirement already satisfied: tensorflow in /opt/conda/lib/python3.10/site-  
packages (2.16.1)  
Requirement already satisfied: keras in /opt/conda/lib/python3.10/site-packages  
(3.3.3)  
Requirement already satisfied: absl-py>=1.0.0 in /opt/conda/lib/python3.10/site-  
packages (from tensorflow) (1.4.0)  
Requirement already satisfied: astunparse>=1.6.0 in  
/opt/conda/lib/python3.10/site-packages (from tensorflow) (1.6.3)  
Requirement already satisfied: flatbuffers>=23.5.26 in  
/opt/conda/lib/python3.10/site-packages (from tensorflow) (24.3.25)  
Requirement already satisfied: gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 in  
/opt/conda/lib/python3.10/site-packages (from tensorflow) (0.5.4)  
Requirement already satisfied: google-pasta>=0.1.1 in  
/opt/conda/lib/python3.10/site-packages (from tensorflow) (0.2.0)  
Requirement already satisfied: h5py>=3.10.0 in /opt/conda/lib/python3.10/site-  
packages (from tensorflow) (3.11.0)  
Requirement already satisfied: libclang>=13.0.0 in  
/opt/conda/lib/python3.10/site-packages (from tensorflow) (18.1.1)  
Requirement already satisfied: ml-dtypes~=0.3.1 in  
/opt/conda/lib/python3.10/site-packages (from tensorflow) (0.3.2)  
Requirement already satisfied: opt-einsum>=2.3.2 in  
/opt/conda/lib/python3.10/site-packages (from tensorflow) (3.3.0)  
Requirement already satisfied: packaging in /opt/conda/lib/python3.10/site-  
packages (from tensorflow) (21.3)  
Requirement already satisfied:  
protobuf!=4.21.0,!4.21.1,!4.21.2,!4.21.3,!4.21.4,!4.21.5,<5.0.0dev,>=3.20.3  
in /opt/conda/lib/python3.10/site-packages (from tensorflow) (3.20.3)  
Requirement already satisfied: requests<3,>=2.21.0 in  
/opt/conda/lib/python3.10/site-packages (from tensorflow) (2.32.3)  
Requirement already satisfied: setuptools in /opt/conda/lib/python3.10/site-  
packages (from tensorflow) (70.0.0)  
Requirement already satisfied: six>=1.12.0 in /opt/conda/lib/python3.10/site-  
packages (from tensorflow) (1.16.0)  
Requirement already satisfied: termcolor>=1.1.0 in  
/opt/conda/lib/python3.10/site-packages (from tensorflow) (2.4.0)  
Requirement already satisfied: typing-extensions>=3.6.6 in
```

/opt/conda/lib/python3.10/site-packages (from tensorflow) (4.12.2)
 Requirement already satisfied: wrapt>=1.11.0 in /opt/conda/lib/python3.10/site-packages (from tensorflow) (1.16.0)
 Requirement already satisfied: grpcio<2.0,>=1.24.3 in /opt/conda/lib/python3.10/site-packages (from tensorflow) (1.62.2)
 Requirement already satisfied: tensorboard<2.17,>=2.16 in /opt/conda/lib/python3.10/site-packages (from tensorflow) (2.16.2)
 Requirement already satisfied: tensorflow-io-gcs-filesystem>=0.23.1 in /opt/conda/lib/python3.10/site-packages (from tensorflow) (0.37.0)
 Requirement already satisfied: numpy<2.0.0,>=1.23.5 in /opt/conda/lib/python3.10/site-packages (from tensorflow) (1.26.4)
 Requirement already satisfied: rich in /opt/conda/lib/python3.10/site-packages (from keras) (13.7.1)
 Requirement already satisfied: namex in /opt/conda/lib/python3.10/site-packages (from keras) (0.0.8)
 Requirement already satisfied: optree in /opt/conda/lib/python3.10/site-packages (from keras) (0.11.0)
 Requirement already satisfied: wheel<1.0,>=0.23.0 in /opt/conda/lib/python3.10/site-packages (from astunparse>=1.6.0->tensorflow) (0.43.0)
 Requirement already satisfied: charset-normalizer<4,>=2 in /opt/conda/lib/python3.10/site-packages (from requests<3,>=2.21.0->tensorflow) (3.3.2)
 Requirement already satisfied: idna<4,>=2.5 in /opt/conda/lib/python3.10/site-packages (from requests<3,>=2.21.0->tensorflow) (3.7)
 Requirement already satisfied: urllib3<3,>=1.21.1 in /opt/conda/lib/python3.10/site-packages (from requests<3,>=2.21.0->tensorflow) (1.26.18)
 Requirement already satisfied: certifi>=2017.4.17 in /opt/conda/lib/python3.10/site-packages (from requests<3,>=2.21.0->tensorflow) (2024.7.4)
 Requirement already satisfied: markdown>=2.6.8 in /opt/conda/lib/python3.10/site-packages (from tensorboard<2.17,>=2.16->tensorflow) (3.6)
 Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /opt/conda/lib/python3.10/site-packages (from tensorboard<2.17,>=2.16->tensorflow) (0.7.2)
 Requirement already satisfied: werkzeug>=1.0.1 in /opt/conda/lib/python3.10/site-packages (from tensorboard<2.17,>=2.16->tensorflow) (3.0.3)
 Requirement already satisfied: pyarsing!=3.0.5,>=2.0.2 in /opt/conda/lib/python3.10/site-packages (from packaging->tensorflow) (3.1.2)
 Requirement already satisfied: markdown-it-py>=2.2.0 in /opt/conda/lib/python3.10/site-packages (from rich->keras) (3.0.0)
 Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /opt/conda/lib/python3.10/site-packages (from rich->keras) (2.18.0)
 Requirement already satisfied: mdurl~=0.1 in /opt/conda/lib/python3.10/site-packages (from markdown-it-py>=2.2.0->rich->keras) (0.1.2)

Requirement already satisfied: MarkupSafe>=2.1.1 in
/opt/conda/lib/python3.10/site-packages (from
werkzeug>=1.0.1->tensorboard<2.17,>=2.16->tensorflow) (2.1.5)
Note: you may need to restart the kernel to use updated packages.

```
[4]: import os
import pandas as pd
import numpy as np
import cv2
from sklearn.model_selection import train_test_split

base_path = '/kaggle/input/indian-emergency-vehicles-dataset'
folders = ['train', 'test', 'valid']

data = []
labels = []

desired_classes = ['ambulance', 'firetruck', 'police vehicle']

for folder in folders:
    csv_path = os.path.join(base_path, folder, '_classes.csv')
    df = pd.read_csv(csv_path)
    df.columns = df.columns.str.strip()
    filtered_df = df[['filename'] + desired_classes]

    # Filter rows where any desired class has a value of 1
    filtered_df = filtered_df[filtered_df[desired_classes].any(axis=1)]

    for index, row in filtered_df.iterrows():
        filename = row['filename']
        image_path = os.path.join(base_path, folder, filename)
        image = cv2.imread(image_path)

        if image is not None:
            image = cv2.resize(image, (128, 128))
            data.append(image)

            label_vector = [int(row[class_name] == 1) for class_name in
desired_classes]
            labels.append(label_vector)

data = np.array(data)
data = data.astype('float32') / 255.0
labels = np.array(labels, dtype=np.float32)

# Split train and validation data
```

```
X_train, X_val, y_train, y_val = train_test_split(data, labels, test_size=0.2,
↪random_state=42)

print(f"Training data shape: {X_train.shape}, Training labels shape: {y_train.
↪shape}")
print(f"Validation data shape: {X_val.shape}, Validation labels shape: {y_val.
↪shape}")
```

Training data shape: (4166, 128, 128, 3), Training labels shape: (4166, 3)
 Validation data shape: (1042, 128, 128, 3), Validation labels shape: (1042, 3)

```
[22]: from tensorflow import keras
from keras.models import Sequential
from keras.layers import Conv2D, MaxPool2D, Flatten, Dense, Dropout
```

```
[23]: model = Sequential()

model.add(Conv2D(32,(3,3), activation='relu', input_shape=(128,128,3)))
model.add(MaxPool2D(pool_size=(2,2)))

model.add(Conv2D(64,(3,3), activation='relu'))
model.add(MaxPool2D(pool_size=(2,2)))

model.add(Conv2D(128,(3,3), activation='relu'))
model.add(MaxPool2D(pool_size=(2,2)))

model.add(Flatten())

model.add(Dense(128,activation='relu'))
model.add(Dropout(0.25))
model.add(Dense(64,activation='relu'))
model.add(Dropout(0.25))
model.add(Dense(32,activation='relu'))

model.add(Dense(3, activation='sigmoid'))
```

/opt/conda/lib/python3.10/site-packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
[24]: model.summary()
```

Model: "sequential_6"

Layer (type)	Output Shape	Param #
conv2d_14 (Conv2D)	(None, 126, 126, 32)	896
max_pooling2d_13 (MaxPooling2D)	(None, 63, 63, 32)	0
conv2d_15 (Conv2D)	(None, 61, 61, 64)	18,496
max_pooling2d_14 (MaxPooling2D)	(None, 30, 30, 64)	0
conv2d_16 (Conv2D)	(None, 28, 28, 128)	73,856
max_pooling2d_15 (MaxPooling2D)	(None, 14, 14, 128)	0
flatten_3 (Flatten)	(None, 25088)	0
dense_9 (Dense)	(None, 128)	3,211,392
dropout (Dropout)	(None, 128)	0
dense_10 (Dense)	(None, 64)	8,256
dropout_1 (Dropout)	(None, 64)	0
dense_11 (Dense)	(None, 32)	2,080
dense_12 (Dense)	(None, 3)	99

Total params: 3,315,075 (12.65 MB)

Trainable params: 3,315,075 (12.65 MB)

Non-trainable params: 0 (0.00 B)

```
[25]: model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

```
[26]: model.fit(X_train, y_train, validation_data=(X_val, y_val), epochs=20,
↳ batch_size=32 )
```

Epoch 1/20

131/131 0s 45ms/step -
accuracy: 0.5531 - loss: 0.5953

I0000 00:00:1729776120.059690 357 asm_compiler.cc:369] ptxas warning :

Registers are spilled to local memory in function
'input_reduce_select_fusion_2', 72 bytes spill stores, 72 bytes spill loads

```
131/131          13s 55ms/step -  
accuracy: 0.5533 - loss: 0.5951 - val_accuracy: 0.6046 - val_loss: 0.5059  
Epoch 2/20  
131/131          2s 17ms/step -  
accuracy: 0.6167 - loss: 0.5042 - val_accuracy: 0.6324 - val_loss: 0.4880  
Epoch 3/20  
131/131          2s 17ms/step -  
accuracy: 0.6761 - loss: 0.4375 - val_accuracy: 0.6756 - val_loss: 0.4403  
Epoch 4/20  
131/131          2s 16ms/step -  
accuracy: 0.7211 - loss: 0.3876 - val_accuracy: 0.7370 - val_loss: 0.3755  
Epoch 5/20  
131/131          2s 16ms/step -  
accuracy: 0.8036 - loss: 0.2957 - val_accuracy: 0.7812 - val_loss: 0.3362  
Epoch 6/20  
131/131          2s 16ms/step -  
accuracy: 0.8656 - loss: 0.2272 - val_accuracy: 0.8081 - val_loss: 0.3243  
Epoch 7/20  
131/131          2s 17ms/step -  
accuracy: 0.9166 - loss: 0.1460 - val_accuracy: 0.8100 - val_loss: 0.3526  
Epoch 8/20  
131/131          2s 16ms/step -  
accuracy: 0.9385 - loss: 0.1058 - val_accuracy: 0.8321 - val_loss: 0.3548  
Epoch 9/20  
131/131          2s 17ms/step -  
accuracy: 0.9561 - loss: 0.0835 - val_accuracy: 0.8321 - val_loss: 0.4237  
Epoch 10/20  
131/131          2s 16ms/step -  
accuracy: 0.9774 - loss: 0.0506 - val_accuracy: 0.8436 - val_loss: 0.4940  
Epoch 11/20  
131/131          2s 17ms/step -  
accuracy: 0.9782 - loss: 0.0465 - val_accuracy: 0.8273 - val_loss: 0.4113  
Epoch 12/20  
131/131          2s 17ms/step -  
accuracy: 0.9741 - loss: 0.0504 - val_accuracy: 0.8301 - val_loss: 0.6025  
Epoch 13/20  
131/131          2s 17ms/step -  
accuracy: 0.9359 - loss: 0.1292 - val_accuracy: 0.8464 - val_loss: 0.4094  
Epoch 14/20  
131/131          2s 17ms/step -  
accuracy: 0.9861 - loss: 0.0383 - val_accuracy: 0.8445 - val_loss: 0.4957  
Epoch 15/20  
131/131          2s 17ms/step -  
accuracy: 0.9872 - loss: 0.0281 - val_accuracy: 0.8292 - val_loss: 0.5637
```

```

Epoch 16/20
131/131          2s 17ms/step -
accuracy: 0.9913 - loss: 0.0198 - val_accuracy: 0.8445 - val_loss: 0.5775
Epoch 17/20
131/131          2s 17ms/step -
accuracy: 0.9830 - loss: 0.0339 - val_accuracy: 0.8407 - val_loss: 0.5109
Epoch 18/20
131/131          2s 17ms/step -
accuracy: 0.9919 - loss: 0.0163 - val_accuracy: 0.8244 - val_loss: 0.5181
Epoch 19/20
131/131          2s 17ms/step -
accuracy: 0.9885 - loss: 0.0276 - val_accuracy: 0.8244 - val_loss: 0.6702
Epoch 20/20
131/131          2s 17ms/step -
accuracy: 0.9620 - loss: 0.0779 - val_accuracy: 0.8225 - val_loss: 0.5198

```

[26]: <keras.src.callbacks.history.History at 0x7e76c068b490>

```

[53]: import numpy as np
import cv2
import random
import matplotlib.pyplot as plt

def predict_random_image(model, base_path, folder):
    csv_path = os.path.join(base_path, folder, '_classes.csv')
    df = pd.read_csv(csv_path)
    random_index = random.randint(0, len(df) - 1)
    random_row = df.iloc[random_index]

    filename = random_row['filename'] # Assuming the first column is 'filename'
    image_path = os.path.join(base_path, folder, filename)
    image = cv2.imread(image_path)
    image_resized = cv2.resize(image, (128, 128)) # Resize to match model input
    image_array = np.array(image_resized, dtype='float32') / 255.0
    image_array = np.expand_dims(image_array, axis=0) # Add batch dimension

    predictions = model.predict(image_array)[0]

    class_names = ['ambulance', 'firetruck', 'police vehicle']
    predicted_classes = [class_names[i] for i in range(len(predictions)) if
↳ predictions[i] > 0.5]

    plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
    plt.axis('off')
    plt.title(f'Filename: {filename}\nPredicted Classes: {predicted_classes}')
    plt.show()

```

```
return predicted_classes

predicted_classes = predict_random_image(model, base_path, 'test')
```

1/1

0s 20ms/step

Filename: ambulance-9-_jpg.rf.740a12448a61433a6e9af32ed096e53c.jpg
Predicted Classes: ['ambulance']



Prediction on custom image

```
[58]: def predict_custom_image(model, image_path):

    image = cv2.imread(image_path)
    image_resized = cv2.resize(image, (128, 128))
    image_array = np.array(image_resized, dtype='float32') / 255.0
    image_array = np.expand_dims(image_array, axis=0)

    predictions = model.predict(image_array)[0]
    predicted_classes = [desired_classes[i] for i in range(len(predictions)) if
↳ predictions[i] > 0.5]

    plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
    plt.axis('off')
    plt.title(f'Predicted Classes: {predicted_classes}')
    plt.show()
```

```
return predicted_classes
```

```
custom_image_path = '/kaggle/input/ambulance/firetruck11.jpeg'  
predicted_classes = predict_custom_image(model, custom_image_path)
```

1/1

0s 18ms/step

Predicted Classes: ['firetruck']



Model Saving

```
[55]: model.save('final_model.h5')
```